Micro-blogging Social Network

Project Requirements Document and Technical Documentation

Project 1 · MCA Group Project · Version 1.0 · 3 November 2026

This document covers the project overview, product requirements, team roles, architecture and technology stack, database schema and API contracts, GitHub workflow and AGENTS.md, testing and security, definition of done, the 15-day roadmap, and viva preparation. It contains documentation only; no application code. Items not in the official requirements are marked **[Optional]** or **[Open Decision]**.

Contents

1. Project Overview **3**

2. Product Requirements Document (PRD) **4**

2A. Product Features . . . 4
2B. User Journeys . . . 5
2C. Functional Requirements . . . 6
2D. Non-Functional Requirements . . . 7
2E. Acceptance Criteria . . . 8
2F. Edge Cases . . . 9

3. Team Roles and Ownership **9**

4. System Architecture and Technology Stack **10**

4.1 Recommended Stack . . . 10
4.2 Architecture Diagram . . . 11
4.3 Explanations . . . 11

5. Database Schema and API Contracts **13**

5.1 Tables . . . 13
5.2 Relationships . . . 15
5.3 Example Records . . . 15
5.4 Redis Keys . . . 15
5.5 REST API Contract . . . 16
5.6 Post Object, Cursor and Socket.IO Contract . . . 18

6. GitHub Repository and CI/CD Integration **18**

6.1 Monorepo Structure (pnpm workspaces) . . . 18
6.2 GitHub Workflow . . . 19
6.3 AGENTS.md . . . 19

7. Testing and Security **20**

7.1 Testing Plan . . . 20
7.2 Security Controls . . . 21

8. Definition of Done **21**

9. 15-Day Implementation Roadmap **22**

10. Beginner-Friendly MCA Viva Preparation **24**

10.1 Core Explanations . . . 24
10.2 Thirty Likely Viva Questions . . . 25
10.3 Individual Contribution Template . . . 26

Assumptions **27**

Decisions the Team Must Approve Before Development **27**

1. Project Overview

1.1 Introduction

This project is a micro-blogging social network in the style of X (Twitter) or Threads. Users write short posts (with images), follow other users, and read a fast home feed made of posts from the people they follow. They can like, repost and reply, use hashtags and mentions, and receive notifications. An AI model checks every new post for abuse.

1.2 Problem Statement

On a social network, reading is far more common than writing. If the home feed is built by querying the database every time a user opens the app, the app becomes slow as the number of users and follows grows. Users also need safety controls (private accounts, block, mute, moderation) and live updates without refreshing the page.

1.3 Proposed Solution

- Build each follower's feed ahead of time (**fan-out on write**). When someone posts, the post ID is pushed into every follower's Redis feed list. Reading the feed then needs one fast Redis call.
- Use PostgreSQL as the source of truth and Redis as a speed layer.
- Use Socket.IO for live counts and live notifications.
- Use background jobs for slow work (fan-out, trending, AI moderation).

1.4 Project Objectives

1. Deliver all 15 official day-wise outcomes.
2. Keep follower, following and engagement counts correct under concurrent requests.
3. Serve the home feed from Redis with cursor-based pagination.
4. Update likes and reposts live through Socket.IO.
5. Check every post with an LLM and queue flagged posts for review.
6. Add rate limiting, tests, a README, deployment and a demo.

1.5 Target Users

User	Need
Reader	Fast feed, search, hashtags
Creator	Post, receive engagement, grow followers
Privacy-conscious user	Private account, block, mute
Moderator	Review flagged posts
MCA team / evaluator	Clean architecture and demonstrable modules

1.6 Project Scope

In scope (official requirements)	Out of scope (not required)
Everything in the Day 1–15 plan	Direct messages, video, polls
	Ads and a recommendation algorithm ("For You" feed)
	Mobile apps, email verification, OAuth login

1.7 Expected Outcomes

A deployed web application, a GitHub repository with CI, a README, a demo, and every team member able to explain their own module in the viva.

1.8 MVP Definition

The MVP is the smallest version that still counts as a working micro-blog with a fast feed (Days 1–6, plus likes from Day 7): register and log in, edit profile, post text, reply, follow and unfollow with correct counts, home feed (database version first, then the Redis fan-out version), and like. Everything after that adds depth, and the official plan still requires all of it.

2. Product Requirements Document (PRD)

2A. Product Features

P0 = core; the app is broken without it. **P1** = official feature that builds on the core. **P2** = official but lowest risk to defer if time runs out. All features are required by the official plan; priority only decides build order under time pressure.

#	Feature (Day)	How it works	Depends on	Pri.
F1	Foundation (1)	TypeScript monorepo, database schema and migrations, Redis in Docker	–	P0
F2	Auth and profiles (2)	Register/login, hashed passwords, token auth. Profile with avatar, bio and unique username	F1	P0
F3	Posts and replies (3)	Text up to 280 characters plus up to 4 images. A reply is a post with `parent_id`. Threads are loaded by `root_id`	F2	P0
F4	Follow system (4)	Follow/unfollow. Counters change atomically in the same database transaction as the follow row	F2	P0
F5	Home feed (5)	Posts from followed users, newest first, cursor pagination, infinite scroll	F3, F4	P0
F6	Fast feed (6)	On a new post, a background job pushes the post ID into each follower's Redis feed. Read = ID list from Redis, then fetch posts by ID	F5, Redis	P0
F7	Likes and reposts (7)	Idempotent like/repost. Counts pushed to clients via Socket.IO	F3, Socket.IO	P0
F8	Hashtags and mentions (8)	Parse `#tag` and `@user` from text on post creation. Hashtag page lists posts	F3	P1
F9	Trending (9)	A repeating job counts hashtags from the last 24 h and caches the top list in Redis every ~5 minutes	F8, jobs	P1
F10	Private accounts (10)	Private profile hides posts from non-followers. Following needs approval through a follow request	F4	P1
F11	Notifications (11)	Created for like, reply, mention, follow. Grouped on read ("Rahul and 5 others liked…")	F7, F8, F4	P1
F12	AI moderation (12)	A job sends new post text to an LLM. Flagged posts are hidden and placed in a moderation queue	F3, jobs	P1
F13	Search, block, mute (13)	Search users/posts. Block = mutual invisibility and no interaction. Mute = hide that user from my feed and notifications	F3, F4	P1
F14	Reliability (14)	Rate limit on posting, automated tests, README	all	P0
F15	Completion (15)	Deployment, demo, individual viva	all	P0

Dependency chain: F1 → F2 → F3 → F4 → F5 → F6 → (F7, F8, F10) → (F9, F11, F12, F13) → F14 → F15.

2B. User Journeys

Journey	Steps
Registration and login	Open app → enter email, username, password → server validates and checks the username is unique → account created and logged in → profile setup (avatar, bio). Later: login → token issued → feed opens.
Creating a post	Click compose → type text, attach images → submit → server validates and checks the rate limit → post saved and shown on my profile → hashtags and mentions parsed → fan-out and AI check run in the background.
Replying to a thread	Open a post → type reply → saved with `parent_id` → thread view shows replies in order → the original author gets a notification.
Following users	Open a profile → Follow → button changes at once and the count updates. If the account is private, the button becomes "Requested".
Viewing the home feed	Open Home → first 20 posts → scroll to the bottom → next page loads using a cursor → repeat.
Liking and reposting	Tap heart/repost → UI updates at once → server stores it once → everyone viewing the post sees the new count live.
Hashtags and mentions	Write "Learning #docker with @rahul" → the hashtag becomes a link to its page → Rahul gets a notification.
Follow requests	A requests B (private) → B sees it under "Requests" → B approves → A is now a follower and B's posts reach A's feed. If B rejects, nothing changes.
Notifications	Bell shows the unread count (live) → open the list → grouped items → click goes to the post or profile → marked read.
Blocking and muting	Profile menu → Block: follows between us are removed and we cannot see or interact with each other. Mute: they stay followed but disappear from my feed and notifications.
Reporting / moderating (report button is [Optional P2])	AI auto-flags a post → it is hidden and added to the queue → a moderator approves (post returns) or removes it. A user "Report" button is optional and feeds the same queue.

2C. Functional Requirements

ID	Requirement (expected system behaviour)	Module
FR-001	System shall register a user with email, username and password	auth
FR-002	Usernames shall be unique, case-insensitive, 3–20 chars, `a-z 0-9 _`	auth / users
FR-003	Passwords shall be stored only as salted hashes	auth
FR-004	System shall issue an access token and a refresh token on login	auth
FR-005	System shall allow logout (refresh token revoked)	auth
FR-006	User shall edit display name (≤50), bio (≤160) and avatar	users
FR-007	Avatar shall be JPEG/PNG/WebP, ≤ 2 MB	users / media
FR-008	System shall show a public profile by username	users
FR-009	User shall create a post of 1–280 chars, or images only	posts
FR-010	A post may include up to 4 images, each ≤ 5 MB	posts / media
FR-011	Whitespace-only posts shall be rejected	posts
FR-012	User shall reply to any visible post; a reply is a post with `parent_id`	posts
FR-013	System shall return a thread (root + replies, paginated)	posts
FR-014	Author shall soft-delete own post; deleted posts vanish from feeds and counts update	posts
FR-015	User shall follow another user (idempotent)	follows
FR-016	User shall unfollow (idempotent)	follows
FR-017	User cannot follow themselves	follows
FR-018	Follower/following counts shall stay correct under concurrent requests	follows
FR-019	System shall list followers/following with cursor pagination	follows
FR-020	Home feed shall contain posts of followed users only, newest first	feed
FR-021	Feed shall use cursor-based pagination (`cursor`, `limit`)	feed
FR-022	Frontend shall load more posts automatically on scroll	web
FR-023	On post creation, the post ID shall be pushed to each follower's Redis feed	feed / jobs
FR-024	Feed reads shall come from Redis; if Redis is down, fall back to PostgreSQL	feed
FR-025	Redis feed shall be capped (e.g., newest 800 IDs per user)	feed
FR-026	Unfollow/block shall stop that user's posts from showing in my feed	feed
FR-027	User shall like/unlike a post; one like per user per post	engagement
FR-028	User shall repost/undo repost; one repost per user per post	engagement
FR-029	Like/repost/reply counts shall be broadcast via Socket.IO to viewers	realtime
FR-030	Socket connections shall be authenticated	realtime
FR-031	System shall parse `#hashtags` (case-insensitive) on post create	hashtags
FR-032	System shall parse `@mentions` and link only existing users	hashtags
FR-033	Hashtag page shall list posts newest first, paginated	hashtags
FR-034	A job shall compute top hashtags of the last 24 h every ~5 min	trending
FR-035	Trending list shall be served from the Redis cache	trending
FR-036	User shall switch account between private and public	users / privacy
FR-037	Following a private user shall create a follow request	follows
FR-038	Private user shall approve/reject requests	follows
FR-039	Non-followers shall not see a private user's posts, followers list or feed entries	privacy
FR-040	Switching private → public shall auto-approve pending requests (decision D10)	privacy
FR-041	Notifications shall be created for like, reply, mention, follow (+ follow request/approval)	notifications
FR-042	No notification for acting on your own content	notifications
FR-043	Notification list shall group by type + post ("Rahul and 5 others liked…")	notifications
FR-044	Unread count shall update live	notifications / realtime
FR-045	User shall mark notifications as read	notifications

ID	Requirement (expected system behaviour)	Module
FR-046	Every new post shall be queued for LLM abuse check	moderation
FR-047	Flagged posts shall be hidden and inserted into the moderation queue	moderation
FR-048	Moderator shall approve or remove queued posts	moderation
FR-049	If the LLM fails, the job shall retry, then mark the post for manual review	moderation
FR-050	Search shall find users (username / display name)	search
FR-051	Search shall find posts by text, excluding blocked, inaccessible and hidden posts	search
FR-052	User shall block/unblock; blocking removes follows both ways	blocks
FR-053	Blocked users cannot view, follow, reply, like, repost or mention each other	blocks
FR-054	User shall mute/unmute; muted users' posts and notifications are hidden for me only	mutes
FR-055	Post creation shall be rate limited per user (e.g., 10 posts/min)	rate-limit
FR-056	Rate-limited requests shall get HTTP 429 with a `Retry-After` header	rate-limit
FR-057	Project shall include automated tests and a README	quality
FR-058	App shall be deployed with a public URL	devops

2D. Non-Functional Requirements

ID	Area	Requirement
NFR-01	Performance	Home feed p95 < 200 ms (Redis path) with 20 items
NFR-02	Performance	Post creation API returns < 300 ms (fan-out and AI run in the background)
NFR-03	Performance	A live count reaches other clients in < 1 s
NFR-04	Scalability	Backend is stateless, so more instances can be added. Socket.IO can use a Redis adapter if scaled
NFR-05	Scalability	Feed lists are capped; indexes on all list queries
NFR-06	Reliability	Redis down → feed falls back to the database. AI down → post still saved, check retried
NFR-07	Reliability	Jobs are retried with backoff; no lost fan-out
NFR-08	Security	See Section 7
NFR-09	Accessibility	Keyboard navigation, alt text for images, labels on buttons/inputs, colour contrast ≥ 4.5:1, `aria-live` for live counts
NFR-10	Maintainability	TypeScript strict mode, shared types package, linting, ≥ 70% coverage on services, PR review before merge
NFR-11	Usability	Works on mobile widths (≥ 360 px)

2E. Acceptance Criteria

Feature	Criteria (Given / When / Then)
Registration	Given username `nick` exists, when I register `Nick`, then I get 409 `USERNAME_TAKEN`. Given valid details, then 201 and a token pair.
Login	Given a wrong password, when I log in, then 401 and no token. The password is never returned.
Post	Given I am logged in, when I post 280 chars, then 201. With 281 chars → 400. With only spaces → 400.
Reply	Given post P, when I reply, then the reply has `parent_id = P`, P's `reply_count` increases by 1, and the thread shows it.
Follow	Given I do not follow U, when I send follow to U twice, then there is 1 follow row and counts changed by exactly 1.
Concurrency	Given 100 users follow U at the same time, then `followers_count` = 100. Given 50 follow/unfollow races by one user, then the count equals the actual number of rows.
Home feed	Given I follow A and B, when I open the feed, then I see only A and B posts, newest first.
Pagination	Given 50 posts, when I fetch with `limit=20` and follow `nextCursor` twice, then I see 50 unique posts with no repeats or gaps, even if new posts arrived between requests.
Fan-out	Given A has 3 followers, when A posts, then within 5 s the post ID exists in 3 Redis feed keys.
Redis failure	Given Redis is stopped, when I open the feed, then I still get posts from the database (slower).
Like	Given I already liked P, when I like again, then the count is unchanged. Another user viewing P sees the count change without refreshing.
Repost	Same as Like for reposts; undo reduces the count by 1.
Hashtags	Given text `Hi #Docker #docker`, then one hashtag `docker` is linked. The hashtag page shows the post.
Mentions	Given `@rahul` exists, then Rahul gets a mention notification. Given `@nobody`, then no error and no link.
Trending	Given a hashtag used within 24 h, then it appears in `/trending/hashtags` after the next job run. Posts older than 24 h do not count.
Private account	Given U is private and I am not approved, when I open U's posts, then I get 403, and U's posts are not in search or hashtag pages for me.
Follow request	Given U is private, when I follow, then status = pending and U is notified. When U approves, I am a follower and counts increase by 1.
Notifications	Given 6 users liked my post, then the list shows one grouped item "X and 5 others liked your post".
AI moderation	Given the LLM flags a post, then it is hidden from feeds/search and appears in the queue. Given the LLM errors, then the post stays visible (pending), retries run, and after final failure it goes to manual review.
Search	Given query `nik`, then users with matching usernames are returned; blocked users are excluded.
Block	Given I block U, then U cannot see my profile or interact with me, and none of our follows remain.
Mute	Given I mute U, then U's posts and notifications do not appear for me, and I still follow U.
Rate limit	Given the limit is 10/min, when I post an 11th time, then 429 with `Retry-After`.

2F. Edge Cases

Case	Expected behaviour
Duplicate username	409. A database unique index on the lowercase username is the final guard, even if two signups race.
Duplicate follow	Idempotent: returns success, no extra row, no count change.
Concurrent follow/unfollow	The counter changes only if the insert/delete actually changed a row (`INSERT … ON CONFLICT DO NOTHING RETURNING`, counter update in the same transaction).
Deleted post	Soft delete. Hidden from feed/search/hashtags; replies show "post deleted". Feed fetch skips missing IDs.
Empty post	400 `VALIDATION_ERROR`.
Very long post	400 at 281+ chars. The frontend counter prevents it.
Duplicate like	The composite primary key stops duplicates. Return 200 with current state.
Private account access	403 for posts, followers and replies unless an approved follower.
Blocked users interacting	403 on follow/like/reply/repost/mention. Hidden from each other in search.
Redis failure	Fall back to the database feed, log a warning, skip cache writes. Rebuild the feed when Redis returns.
Socket disconnection	The client auto-reconnects. On reconnect it refetches current counts via REST.
AI moderation failure	Retry 3× with backoff, then `manual_review`. Never block posting.
Rate-limit violation	429, nothing saved, UI shows "try again in N seconds".
Pagination while new posts arrive	The cursor means "posts older than this ID", not a page number, so new posts never shift pages. New posts appear through a "Show new posts" banner.
Unfollow after fan-out	Old IDs still sit in the feed list, so filter by current follows at read time or purge them.
Author later goes private	Re-check privacy at read time, not only at fan-out.

3. Team Roles and Ownership

Five members (M1–M5) are assumed. Names are placeholders and can be swapped freely. Every module has a primary owner and a reviewer, and every member has an independently demonstrable contribution.

M1: Tech Lead / Auth and Social Graph

Module ownership	Repo, DB schema, auth, profiles, follows, follow requests, private accounts, block/mute rules
Deliverables	Monorepo, migrations, seed data, auth API, profile API, follow API with concurrency test
Responsibilities	Keep the schema consistent, review all DB changes, final say on architecture
Dependencies	None (others depend on M1 first)
GitHub contribution	`packages/db`, `apps/api/src/modules/{auth,users,follows,privacy}`
Viva topics	Schema design, hashing, JWT, unique constraints, transactions, atomic counters
Independent demo	100 concurrent follows through curl/Postman give the correct count.

M2: Feed and Performance

Module ownership	Posts/replies API, home feed, Redis fan-out, pagination, rate limiting
Deliverables	Post/reply API, DB feed, Redis feed, cursor logic, rate limiter, feed load test
Responsibilities	Make the feed fast and correct; explain the fan-out
Dependencies	M1 (users, follows), M4 (parser)
GitHub contribution	`modules/{posts,feed,ratelimit}`, `lib/redis`
Viva topics	Fan-out on write, Redis structures, cursor pagination, rate limiting
Independent demo	Show Redis keys filling after a post, plus a feed latency comparison (DB vs Redis).

M3: Engagement, Realtime and DevOps

Module ownership	Likes, reposts, Socket.IO, notifications, Docker, CI/CD, deployment
Deliverables	Like/repost API, socket server, notification service + grouping, docker-compose, GitHub Actions, live URL
Responsibilities	Real-time behaviour, pipeline health, release
Dependencies	M2 (posts), M1 (users)
GitHub contribution	`modules/{engagement,notifications}`, `realtime/`, `.github/`, `docker-compose.yml`
Viva topics	Socket.IO rooms/events, idempotent likes, notification grouping, CI/CD, Docker
Independent demo	Two browser windows: a like in one updates the other. Show the CI run and the live site.

M4: Content Intelligence

Module ownership	Hashtags, mentions, trending, search, AI moderation, job queue
Deliverables	Parser, hashtag pages, trending job, search API, moderation job + queue API
Responsibilities	Text processing, background jobs, LLM integration
Dependencies	M2 (posts)
GitHub contribution	`modules/{hashtags,search,moderation}`, `jobs/`
Viva topics	Parsing, BullMQ, trending algorithm, LLM moderation flow, search
Independent demo	Post with `#tags @mentions`, watch trending refresh, flag a post and show the moderation queue.

M5: Frontend and UX Lead

Module ownership	React app, all screens, API client, accessibility, UI tests
Deliverables	Login, profile, compose, thread, feed, notification, search and moderation screens
Responsibilities	Consistent UX, infinite scroll, live updates on screen
Dependencies	All API owners; mock data until APIs land
GitHub contribution	`apps/web/**`
Viva topics	Component structure, state/data fetching, infinite scroll, accessibility
Independent demo	Walk through the full UI with infinite scroll and keyboard navigation.

Adjusting the team. With four members, merge M5's DevOps share into M1 and M4's search into M2. With six, split M3 into Engagement and DevOps.

4. System Architecture and Technology Stack

4.1 Recommended Stack

The official requirements fix only TypeScript, Redis, Docker and Socket.IO. Everything else below is a **recommendation to be approved by the team** (see the decision list at the end).

Area	Recommendation	Why (simple)	Alternative
Language	TypeScript (official)	Catches mistakes before running; types shared between frontend and backend	–
Frontend	React + Vite + React Router + TanStack Query	Popular and easy to learn; TanStack Query has built-in infinite scroll support	Next.js (adds server-rendering complexity)
Backend	Node.js + Express + Zod validation	Simplest for beginners, huge tutorial base	Fastify, NestJS (heavier)
Database	PostgreSQL	Strong relations, unique constraints, transactions	MySQL
ORM	Prisma	Readable schema file, migrations, generated TypeScript types	Drizzle
Redis	Official (Docker); client `ioredis`	Fast in-memory store for feeds, cache, rate limits, queues	–
Docker	Official; docker-compose runs Postgres + Redis (+ API)	Same setup on every laptop	–

Area	Recommendation	Why (simple)	Alternative
Real-time	Socket.IO (official)	Auto-reconnect, rooms	–
Background jobs	BullMQ (uses Redis)	Retries, delays, repeatable jobs (trending)	node-cron (no retries)
AI moderation	An LLM behind a small `ModerationProvider` interface [Open Decision: provider]	Provider can be swapped; mockable in tests	–
Authentication	Email + password; short-lived JWT access token + refresh token in httpOnly cookie; argon2 (or bcrypt) hashing	Standard and secure; stateless API	Server sessions in Redis
Image storage	Cloudinary or S3-compatible (e.g., Cloudflare R2) through an upload endpoint; local folder in dev	Files should not be stored in the database	Local disk only (breaks on redeploy)
Testing	Vitest (unit), Supertest (API), Testcontainers or docker-compose test DB (integration), Playwright (E2E), k6 (load), socket.io-client (sockets)	Fast, TypeScript-friendly	Jest
Deployment	Frontend: Vercel/Netlify. API + worker: Render/Railway/Fly.io. DB: Neon/Supabase. Redis: Upstash/Render	Free or cheap tiers	One VPS with docker-compose

4.2 Architecture Diagram

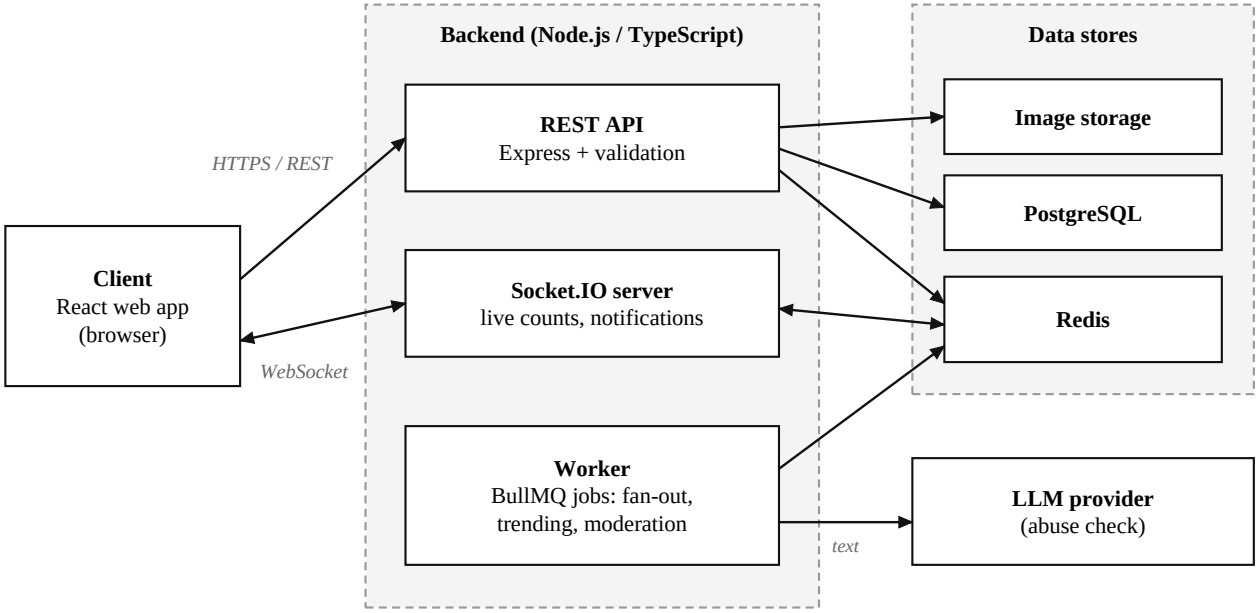


Figure 1. Overall architecture. The browser talks to the backend over REST and WebSocket; the backend uses PostgreSQL, Redis and image storage; the worker also reads and writes PostgreSQL and Redis and calls the LLM provider. Socket.IO receives events from the API through Redis pub/sub.

4.3 Explanations

Overall

The **API** handles requests, the **worker** handles slow jobs, **PostgreSQL** stores everything permanently, and **Redis** stores fast temporary data (feeds, rate-limit counters, trending cache, job queues).

Frontend to backend communication

The web app calls REST endpoints under `/api/v1` with JSON, and the access token goes in the `Authorization` header. It also opens one Socket.IO connection for live events.

Request and response flow (create post)

1. Browser sends `POST /posts` → auth middleware → rate limiter (Redis) → Zod validation.

2. Save the post (and hashtag/mention rows) in a database transaction.
3. Enqueue two jobs: `fanout-post` and `moderate-post`.
4. Return `201` with the post immediately.
5. The worker finishes fan-out and moderation afterwards.

Feed fan-out on write

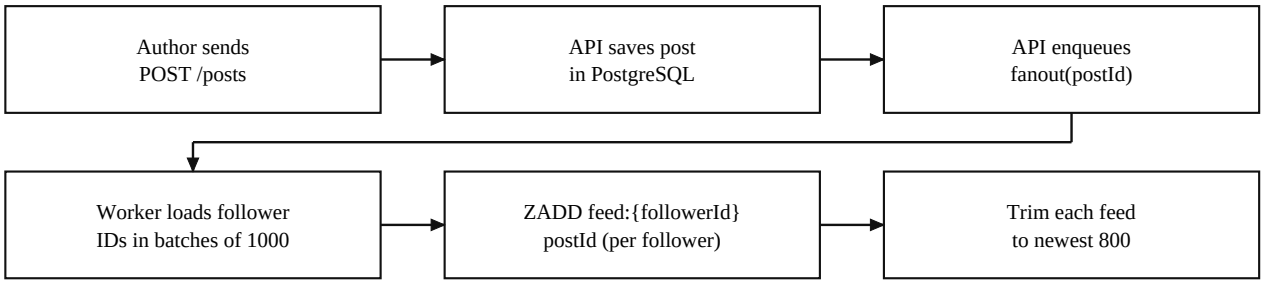


Figure 2. Fan-out on write. The API answers the author as soon as the post is saved; the worker copies the post ID into followers' feeds afterwards.

Simple example. Asha has 3 followers. When Asha posts, the worker adds the post ID to `feed:ravi`, `feed:meera` and `feed:sam`. When Ravi opens Home, the API reads his list of IDs from Redis, loads those posts from the database in one query, and returns them. No join over all of his follows is needed.

Redis feed caching

Key `feed:{userId}` is a **sorted set** (members = post IDs, score = a time-sortable number). A sorted set gives a clean "give me IDs older than X" for cursors. Only IDs are stored, not post bodies, so edits, deletes and count changes stay correct. Filtering (blocked, muted, deleted, private) is re-checked at read time. The official text says "feed lists"; a sorted set is a list ordered by time. **[Open Decision]** if the team prefers a plain Redis List.

Celebrity problem [Optional]

A user with a million followers would make fan-out slow. For the course project, fan-out runs as a chunked background job. A hybrid approach (skip fan-out for huge accounts and merge at read time) is a viva talking point, not a requirement.

Real-time events

After a like/repost/reply commits, the API publishes an event. The socket server emits `post:counts` to room `post:{id}`. New notifications go to room `user:{id}`. If the socket server and API run as separate processes, Redis pub/sub (or the Socket.IO Redis adapter) connects them.

Background jobs

Queues: `fanout`, `moderation`, and `trending` (repeatable every 5 minutes). Each has retries with exponential backoff.

AI moderation workflow

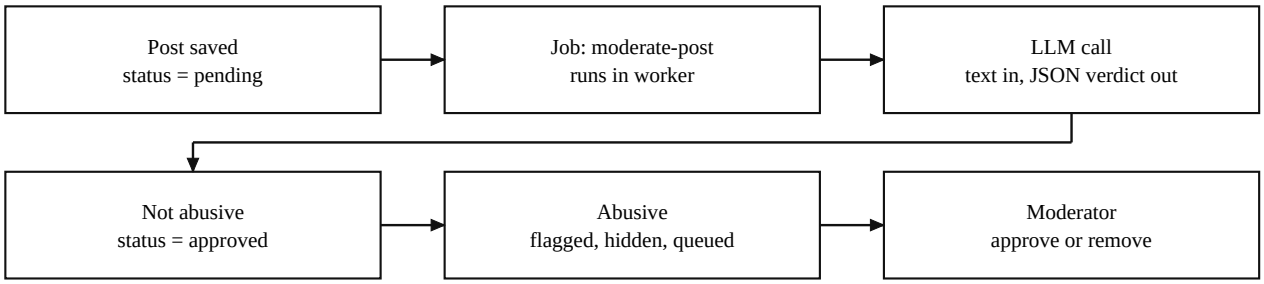


Figure 3. Moderation workflow (main path). If the LLM call fails it is retried 3 times with backoff; if it still fails the post is put in the queue as `manual_review`.

[Open Decision] publish-then-check vs hold-until-checked. Recommended: **publish-then-check**. The post is visible right away and hidden only if flagged. It keeps posting fast, and the LLM never blocks users.

5. Database Schema and API Contracts

Conventions. IDs are UUIDv7 (time-sortable, so `ORDER BY id DESC` equals newest first and cursors stay simple). Timestamps are `timestamptz`. Usernames are stored case-insensitively (`citext`). Soft delete uses `deleted_at`.

5.1 Tables

users

Field	Type	Constraints
id	uuid	PK
username	citext	UNIQUE, NOT NULL, 3–20 chars
email	citext	UNIQUE, NOT NULL
password_hash	text	NOT NULL
display_name	varchar(50)	
bio	varchar(160)	
avatar_url	text	nullable
is_private	boolean	default false
role	enum(user, moderator, admin)	default user
followers_count, following_count, posts_count	int	default 0, CHECK ≥ 0
created_at, deleted_at	timestamptz	

posts

Field	Type	Constraints
id	uuid	PK
author_id	uuid	FK → users, NOT NULL
parent_id	uuid	FK → posts, nullable (reply target)
root_id	uuid	FK → posts, nullable (thread root)
content	varchar(280)	CHECK: content is non-blank OR post has media
media_urls	text[]	max 4 items
like_count, repost_count, reply_count	int	default 0
moderation_status	enum(pending, approved, flagged, removed)	default pending
created_at, deleted_at	timestamptz	

Indexes: (author_id, id DESC); (parent_id, id); (root_id, id); (moderation_status); full-text GIN index on to_tsvector(content) for search.

follows

Field	Type	Constraints
follower_id	uuid	FK → users; part of PK
followee_id	uuid	FK → users; part of PK
created_at	timestamptz	CHECK follower_id ≠ followee_id

PK (follower_id, followee_id). Index (followee_id, created_at DESC).

follow_requests

Field	Type	Constraints
id	uuid	PK
requester_id	uuid	FK → users
target_id	uuid	FK → users
status	enum(pending, approved, rejected)	
created_at, resolved_at	timestamptz	

UNIQUE (requester_id, target_id). Index (target_id, status).

likes / reposts (same shape)

Field	Type	Constraints
user_id	uuid	FK → users; part of PK
post_id	uuid	FK → posts; part of PK
created_at	timestamptz	

PK (user_id, post_id). Index (post_id).

hashtags

Field	Type	Constraints
id	uuid	PK
tag	varchar(100)	UNIQUE (lowercase, no #)
created_at	timestamptz	

post_hashtags

Field	Type	Constraints
post_id	uuid	FK → posts; part of PK
hashtag_id	uuid	FK → hashtags; part of PK
created_at	timestamptz	

PK (post_id, hashtag_id). Indexes (hashtag_id, post_id DESC) and (created_at, hashtag_id) for the 24 h trending query.

mentions

Field	Type	Constraints
post_id	uuid	FK → posts; part of PK
mentioned_user_id	uuid	FK → users; part of PK

PK (post_id, mentioned_user_id).

notifications

Field	Type	Constraints
id	uuid	PK
recipient_id	uuid	FK → users
actor_id	uuid	FK → users
type	enum(like, reply, mention, follow, follow_request, follow_approved)	
post_id	uuid	FK → posts, nullable
read_at	timestamptz	nullable
created_at	timestamptz	

Indexes (recipient_id, created_at DESC) and (recipient_id, type, post_id). One row per event; grouping happens when reading, so an unlike can delete a row cleanly.

blocks / mutes (same shape)

Field	Type	Constraints
blocker_id / muter_id	uuid	FK → users; part of PK
target_id	uuid	FK → users; part of PK
created_at	timestamptz	

PK (blocker_id, target_id). Extra index on target_id for the reverse check.

moderation_queue

Field	Type	Constraints
id	uuid	PK
post_id	uuid	FK → posts, UNIQUE per open item
source	enum(ai, user_report)	user_report is Optional
status	enum(open, approved, removed, manual_review)	
ai_label	varchar(50)	e.g. harassment
ai_score	numeric(4,3)	0–1
reason	text	
reviewed_by	uuid	FK → users, nullable
created_at, reviewed_at	timestamptz	

5.2 Relationships

users 1—N posts; posts 1—N posts (replies, via `parent_id`); users N—N users (follows, blocks, mutes); users N—N posts (likes, reposts); posts N—N hashtags; posts N—N users (mentions); users 1—N notifications; posts 1—N moderation_queue.

5.3 Example Records

Table	Example
users	`{id: u1, username: "asha", is_private: false, followers_count: 3}`
posts	`{id: p1, author_id: u1, parent_id: null, content: "Learning #docker with @rahul", moderation_status: "approved"}`
posts (reply)	`{id: p2, author_id: u2, parent_id: p1, root_id: p1, content: "Nice!"}`
follows	`{follower_id: u2, followee_id: u1}`
post_hashtags	`{post_id: p1, hashtag_id: h1}` where h1.tag = "docker"
notifications	`{recipient_id: u1, actor_id: u2, type: "like", post_id: p1, read_at: null}`
moderation_queue	`{post_id: p9, source: "ai", status: "open", ai_label: "harassment", ai_score: 0.93}`

5.4 Redis Keys

Key	Type	Purpose
`feed:{userId}`	Sorted set	Home feed IDs (capped at 800)
`trending:hashtags`	JSON / sorted set	Cached top 10
`ratelimit:post:{userId}`	Counter + TTL	Post rate limit
`bull:*`	BullMQ internal	Job queues

5.5 REST API Contract

Base path `/api/v1`. Success: `{ "data": …, "meta": { "nextCursor": "…" | null } }`. Error: `{ "error": { "code": "…", "message": "…", "details": … } }`. Auth = `Authorization: Bearer <accessToken>` unless "Public". Common errors on every protected route: `401 UNAUTHORIZED`, `429 RATE_LIMITED`.

Method	Endpoint	Purpose	Auth	Request	Success	Errors
POST	/auth/register	Create account	Public	body: email, username, password	201 {user, accessToken} + refresh cookie	400 VALIDATION_ERROR; 409 USERNAME_TAKEN / EMAIL_TAKEN
POST	/auth/login	Log in	Public	body: email or username, password	200 {user, accessToken}	400; 401 INVALID_CREDENTIALS
POST	/auth/refresh	New access token	Cookie	–	200 {accessToken}	401
POST	/auth/logout	Revoke refresh token	Yes	–	204	401
GET	/auth/me	Current user	Yes	–	200 {user}	401
GET	/users/:username	View profile	Optional	–	200 {profile, relationship}	404; 403 BLOCKED
PATCH	/users/me	Edit profile / privacy	Yes	body: displayName, bio, avatarUrl, isPrivate	200 {user}	400
POST	/media	Upload image	Yes	multipart file	201 {url}	400; 413 FILE_TOO_LARGE; 415 BAD_TYPE
GET	/users/:username/posts	User's posts	Optional	cursor, limit	200 list + nextCursor	403 PRIVATE_ACCOUNT; 404
POST	/posts	Create post	Yes	body: content, mediaUrls[]	201 {post}	400; 429 RATE_LIMITED
GET	/posts/:id	Get one post	Optional	–	200 {post}	403; 404
DELETE	/posts/:id	Soft delete own post	Yes	–	204	403 NOT_OWNER; 404
POST	/posts/:id/replies	Reply	Yes	body: content, mediaUrls[]	201 {post}	400; 403 BLOCKED / PRIVATE; 404; 429
GET	/posts/:id/replies	Thread replies	Optional	cursor, limit	200 list + nextCursor	404
PUT	/users/:id/follow	Follow (or request if private)	Yes	–	200 {status: following | requested}	400 SELF_FOLLOW; 403 BLOCKED; 404
DELETE	/users/:id/follow	Unfollow / cancel request	Yes	–	200 {status: none}	404
GET	/users/:id/followers, /following	Lists	Optional	cursor, limit	200 list	403 PRIVATE_ACCOUNT
GET	/follow-requests	My pending requests	Yes	cursor	200 list	–
POST	/follow-requests/:id/approve, /reject	Resolve request	Yes	–	200	403 NOT_TARGET; 404; 409 ALREADY_RESOLVED
GET	/feed	Home feed	Yes	cursor, limit (≤50, default 20)	200 {data: posts[], meta.nextCursor}	400 BAD_CURSOR
PUT	/posts/:id/like	Like (idempotent)	Yes	–	200 {liked: true, likeCount}	403; 404
DELETE	/posts/:id/like	Unlike	Yes	–	200 {liked: false, likeCount}	404

Method	Endpoint	Purpose	Auth	Request	Success	Errors
PUT / DELETE	/posts/:id/repost	Repost / undo	Yes	–	200 {reposted, repostCount}	403; 404
GET	/hashtags/:tag/posts	Hashtag page	Optional	cursor, limit	200 list	404
GET	/trending/hashtags	Trending (cached)	Public	limit	200 [{tag, count}], updatedAt	–
GET	/notifications	Grouped notifications	Yes	cursor, limit	200 grouped list	–
GET	/notifications/unread-count	Badge count	Yes	–	200 {count}	–
POST	/notifications/read	Mark read	Yes	body: ids[] or all: true	204	400
GET	/search/users	Search users	Yes	q (≥ 2 chars), cursor	200 list	400
GET	/search/posts	Search posts	Yes	q, cursor	200 list	400
PUT / DELETE	/users/:id/block	Block / unblock	Yes	–	200	400 SELF; 404
PUT / DELETE	/users/:id/mute	Mute / unmute	Yes	–	200	400 SELF; 404
GET	/me/blocks, /me/mutes	My lists	Yes	cursor	200 list	–
GET	/moderation/queue	Open queue	Moderator	status, cursor	200 list	403
POST	/moderation/queue/:id/decision	Approve / remove	Moderator	body: decision (approve | remove)	200	403; 404; 409
POST	/posts/:id/report	[Optional P2] user report	Yes	body: reason	201	404; 409 ALREADY_REPORTED
GET	/health	Liveness	Public	–	200 {db, redis}	503

5.6 Post Object, Cursor and Socket.IO Contract

Post object (response): `{ id, author:{id, username, displayName, avatarUrl}, content, mediaUrls, parentId, rootId, likeCount, repostCount, replyCount, viewerHasLiked, viewerHasReposted, hashtags[], mentions[], createdAt }`

Cursor format. `cursor` is an opaque Base64 string wrapping the last post ID. The server returns posts with `id < cursorId`, ordered by `id DESC`.

Socket.IO. Connect with `auth: { token: <accessToken> }`; an invalid token causes the connection to be rejected.

Direction	Event	Payload	Notes
Client → Server	`post:subscribe`	`{postId}`	Joins room `post:{id}`. Rejected if the post is not visible to the user
Client → Server	`post:unsubscribe`	`{postId}`	Leaves the room
Server → Client	`post:counts`	`{postId, likeCount, repostCount, replyCount}`	Sent to the room after changes
Server → Client	`notification:new`	`{notification, unreadCount}`	To room `user:{id}` (joined automatically at connect)
Server → Client	`feed:new`	`{count}`	[Optional] "N new posts" banner
Server → Client	`error`	`{code, message}`	e.g. AUTH_FAILED

6. GitHub Repository and CI/CD Integration

6.1 Monorepo Structure (pnpm workspaces)

```
microblog/
|-- AGENTS.md
|-- README.md
|-- package.json              # workspaces, shared scripts
|-- pnpm-workspace.yaml
|-- tsconfig.base.json
|-- docker-compose.yml        # postgres + redis (+ api, web for full-stack run)
|-- .env.example              # variable names only, no real values
|-- .github/
|   |-- workflows/
|   |   |-- ci.yml            # lint, typecheck, test
|   |   `-- deploy.yml        # deploy on merge to main
|   |-- pull_request_template.md
|   `-- CODEOWNERS
|-- apps/
|   |-- web/                  # React + Vite
|   |   `-- src/{pages,components,features,hooks,api,styles}
|   `-- api/                  # Express + worker + sockets
|       `-- src/
|           |-- modules/{auth,users,posts,follows,feed,engagement,hashtags,
|           |            notifications,moderation,search,blocks}
|           |-- jobs/         # BullMQ queues + workers
|           |-- realtime/     # Socket.IO setup and events
|           |-- lib/{redis.ts,db.ts,logger.ts,errors.ts}
|           |-- middleware/{auth,validate,rateLimit,error}.ts
|           `-- server.ts, worker.ts
|-- packages/
|   |-- shared/               # shared types, Zod schemas, constants, event names
|   `-- db/                   # Prisma schema, migrations, seed
|       |-- prisma/schema.prisma
|       `-- prisma/migrations/
|-- tests/
|   |-- e2e/                  # Playwright
|   `-- load/                 # k6 scripts
`-- docs/                     # PRD, architecture, API, viva notes, diagrams
```

Redis configuration lives in `docker-compose.yml` and `apps/api/src/lib/redis.ts`. Database configuration lives in `packages/db` (schema) and `.env`.

6.2 GitHub Workflow

Step	How
Create repo	M1 creates `microblog` on GitHub (private or public as the college allows) with a README and a Node `.gitignore`.
Collaborator access	Settings → Collaborators → add the 4 other members (Write role). M1 is admin.
Cloning	`git clone <url>` then `pnpm install`.
Feature branches	Never work on `main`. Use `feature/<module>-<short-name>`, e.g. `feature/follows-concurrency`. Also `fix/…`, `docs/…`, `test/….`
Commit messages	Conventional style: `feat(follows): make counters atomic`, `fix(feed): skip deleted posts`, `test(auth): add duplicate username test`, `docs: update API table`.
Pull requests	Small PRs (one feature). Fill the template: what, why, how tested, screenshots, linked day/task.
Code review	At least 1 approval from the module reviewer (CODEOWNERS), plus M1 for DB or architecture changes. Reviewer checks: matches PRD/API doc, tests present, no secrets.
Merging	"Squash and merge" after CI is green and the PR is approved. Delete the branch.
Branch protection (main)	Require PR, 1 approval, required status checks (lint, typecheck, test), no force-push, no direct commits, branch up to date before merge.
CI checks	On every PR: install → lint → typecheck → unit tests → integration tests (Postgres + Redis as service containers) → build.
Deployment workflow	On merge to `main`: build → run migrations → deploy API/worker → deploy web → smoke test `/health`. Secrets live in GitHub Actions Secrets and host settings.

6.3 AGENTS.md

Copy the following file to the repository root (`AGENTS.md`).

```
# AGENTS.md - Instructions for AI Coding Agents

Project: Micro-blogging Social Network (X/Threads-style app with a fast Redis home feed).
This is an MCA group project with five contributors. Read this file fully before making changes.

## 1. Source of truth
- Approved docs live in /docs (PRD, architecture, DB schema, API contract).
- Never invent APIs, endpoints, database tables, columns, events, or environment variables.
  If something is missing from the docs, STOP and ask, or propose it clearly as an open question.
- Do not add features outside the official scope (Day 1-15 plan). Optional ideas must be labelled
    [Optional].
## 2. Architecture (do not change silently)
- Monorepo (pnpm): apps/web (React + Vite), apps/api (Express + Socket.IO + BullMQ worker),
  packages/shared (types + Zod schemas), packages/db (Prisma schema + migrations).
- PostgreSQL is the source of truth. Redis is used for feeds, rate limits, trending cache, and
    job queues.
- Feed uses fan-out on write: a background job pushes post IDs into feed:{userId}.
- If a change needs a different library, database, or pattern, explain why and get approval
    first.

## 3. Language and style
- TypeScript only, strict mode. No "any" unless commented with a reason. No JavaScript files.
- Reuse types and Zod schemas from packages/shared. Never duplicate request/response types.
- Follow the ESLint + Prettier config. camelCase for variables, PascalCase for types/components,
  snake_case for database columns, kebab-case for file names.
- Keep functions small. Put business logic in services, not in route handlers.
- Validate every request body/query with Zod. Return the standard error shape { error: { code,
    message } }.

## 4. Scope discipline
- Work only inside the module you were asked to change.
- Do NOT modify unrelated modules, shared config, already-merged migrations, CI files, or other
    people's
  tests unless the task requires it, and then explain why first.
- Never edit an existing migration. Create a new one.
```

```
- Respect the existing folder structure. Do not move or rename files without approval.

## 5. Security rules
- Never print, commit, or hardcode secrets, tokens, API keys, or passwords. Use environment
    variables;
  update .env.example with names only.
- Never log passwords, tokens, or full request bodies of auth routes.
- Use parameterized queries / Prisma only. No string-built SQL.
- Escape/sanitize user content on output. Never use dangerouslySetInnerHTML with user text.
- Enforce authorization on the server: private accounts, blocks, mutes, ownership, moderator
    role.
## 6. Data correctness
- Follower counts, like counts, and similar counters must be updated atomically in the same
    transaction
  as the row insert/delete, and only when a row actually changed.
- Likes, reposts, and follows must be idempotent.
- Feed pagination must use cursors, never page numbers or OFFSET.

## 7. Testing
- Every new feature or bug fix includes tests (unit for logic, integration/API for endpoints).
- Run "pnpm lint && pnpm typecheck && pnpm test" before proposing a commit. Do not disable or
    delete
  failing tests; fix the cause.
- Mock the LLM provider in tests. Never call real paid APIs in tests.

## 8. Git workflow
- Never commit directly to main. Work on feature/<module>-<name> branches.
- Use Conventional Commits (feat(feed): ...). Keep PRs small and focused, one feature each.
- Fill in the PR template. Do not merge your own PR.

## 9. Communication
- Before changing important files (schema, auth, feed logic, CI, shared types, package.json),
  explain what you will change and why, and wait for confirmation.
- After changes, summarize: files touched, behaviour change, tests added, anything left undone.
- Prefer simple, readable code. Add short comments for non-obvious logic (this is a learning
    project).

## 10. Definition of done (short)
Code matches the API contract, tests pass, lint/typecheck pass, docs updated, no secrets, PR
  reviewed.
```

7. Testing and Security

7.1 Testing Plan

Type	Tool	What to test	Owner
Unit	Vitest	Hashtag/mention parser, cursor encode/decode, grouping logic, validators, rate-limit logic	each module owner
Integration	Vitest + real Postgres/Redis (docker)	Services with the DB: follow transaction, fan-out writes Redis keys, trending query	M1, M2, M4
API	Supertest	Each endpoint: success, validation errors, 401/403/404/429	all
Frontend	Vitest + Testing Library; Playwright for E2E	Components, infinite scroll, forms; E2E: register → post → follow → feed	M5
Feed performance	k6	1,000-follower fan-out time; p95 feed latency, Redis vs DB path	M2
Concurrent requests	Script firing 100 parallel requests	Follow/unfollow/like: counts equal row counts, no duplicates	M1, M3
Socket.IO	`socket.io-client` in tests	Auth rejected without token; two clients receive `post:counts`; reconnect refetch	M3
Security	Manual checklist + `npm audit` + OWASP ZAP baseline (optional)	See 7.2	all

7.2 Security Controls

Topic	Control
Password security	argon2/bcrypt with salt; minimum 8 characters; never logged or returned; generic login error message.
Authentication and authorization	Short-lived access token (~15 min); refresh token in an httpOnly + Secure + SameSite cookie with rotation on refresh. Ownership checks on delete/edit; role check on moderation.
Input validation	Zod on every body/query/param; length limits; file type and size checks (verify MIME type, not just the extension).
XSS prevention	React escapes by default; never render raw HTML; mentions/hashtags rendered as components; Content-Security-Policy via Helmet.
Rate limiting	Redis counters: posting limit (official), plus a stricter limit on login to slow brute force.
SQL/NoSQL injection	Prisma parameterized queries; Redis keys built only from validated IDs; no `eval` of user input.
Private account authorization	One shared `canView(viewer, author)` function used by profile, posts, replies, search, hashtags, feed read and sockets.
Block and mute enforcement	Enforced in the API and when reading the feed; tests prove blocked users get 403 and are absent from search.
AI moderation failure handling	Retry, then manual review; never lose the post; log an alert. The LLM prompt treats post text as data, so a post saying "ignore your instructions" cannot change the verdict. Output is kept to a strict JSON schema.
Secrets management	`.env` is git-ignored, `.env.example` is committed, secrets live in GitHub Actions / host dashboard, rotate if leaked, optional GitHub secret scanning.
Other	CORS allow-list, Helmet headers, HTTPS in production, dependency audits in CI.

8. Definition of Done

Area	Checklist
Feature completion	[ ] Matches PRD/FR IDs [ ] Acceptance criteria demonstrated [ ] Edge cases handled [ ] Errors use the standard format [ ] Uses shared types [ ] Frontend and backend connected
Pull request approval	[ ] Small and focused [ ] Template filled [ ] CI green [ ] 1+ approval (M1 for DB/architecture) [ ] No secrets or debug code [ ] Squash merged, branch deleted
Testing	[ ] Unit + API tests added [ ] Concurrency test where counters are involved [ ] Coverage not lowered [ ] Manually tried in the browser
Integration	[ ] Works with Redis and Postgres in docker-compose [ ] Works after pulling latest `main` [ ] Socket events verified with two browsers
Documentation	[ ] API doc / Swagger updated [ ] README steps still work on a clean clone [ ] `.env.example` updated [ ] Diagrams updated if the design changed
Deployment	[ ] Migrations applied [ ] Env vars set on host [ ] `/health` shows DB and Redis OK [ ] Worker running [ ] Image upload works in production
Final demonstration	[ ] Script rehearsed [ ] Seed data (users, follows, posts, a flagged post) [ ] Backup screenshots/video [ ] Everyone can explain their module [ ] Repo clean with visible commits from all members

9. 15-Day Implementation Roadmap

The official order is kept unchanged. Frontend (M5) builds each day's screen against the agreed API contract (mock data if the API is not ready). Each member writes tests for their own work every day. Each day depends only on earlier days. If a day slips, the P0 chain (Days 1–7, 14, 15) is protected first.

Day	Objective and tasks	Modules	Deliverables	Depends on	Testing	Team
1	**Foundation.** Create repo, pnpm workspace, TS config, lint/format, Prisma schema (users, posts, follows, likes, reposts, hashtags, notifications; other Section 5 tables stubbed or added), docker-compose with Redis (and Postgres), health endpoint, CI skeleton	repo, db, infra	`docker compose up` + migration + seed work; CI green on a trivial test	–	Migration applies on a clean DB; health test	M1 repo/schema; M2 reviews schema; M3 docker/CI; M4 shared type stubs; M5 Vite app shell
2	**Auth and profiles.** Register/login/logout/refresh, unique usernames, profile view/edit, avatar upload	auth, users, media	Auth + profile APIs; login/profile pages	D1	Duplicate-username test (incl. race), wrong password, validation	M1 API; M3 media storage; M5 screens; M2/M4 route-guard middleware tests
3	**Posts and replies.** Create post (text + images), delete, replies with `parent_id`/`root_id`, thread view	posts, media	Post/reply APIs; compose box; thread page	D2	Length/empty/image tests, reply counts, thread order	M2 API; M5 UI; M3 image pipeline; M4 plans parser hook; M1 reviews
4	**Follow system.** Follow/unfollow, atomic counters, followers/following lists	follows	Follow API; follow button; lists	D2	**Concurrency test:** 100 parallel follows; duplicate follow; self-follow	M1 API; M5 UI; M3 concurrency script harness
5	**Home feed (DB).** Query posts from followed users, newest first, cursor pagination, infinite scroll	feed	`/feed` API; infinite scroll page	D3, D4	Cursor correctness: no duplicates/gaps with inserts mid-scroll	M2 API; M5 UI; M1 index tuning; M4 seeds 1,000 posts
6	**Fast feed.** BullMQ `fanout` job, Redis feed sorted sets, read from Redis, DB fallback, cap list size	feed, jobs, redis	Redis feed live; fallback working	D5	Keys created for all followers; Redis-down fallback; k6 latency comparison	M2 lead; M4 BullMQ base; M3 monitors Redis/compose; M1 purge on unfollow
7	**Likes and reposts + Socket.IO.** Idempotent like/repost, count updates, socket auth/rooms, `post:counts`	engagement, realtime	Like/repost APIs; live counts in UI	D3	Duplicate like, concurrent likes, two-client socket test	M3 lead; M5 live UI; M2 feed counts; M1 socket auth review
8	**Hashtags and mentions.** Parser, tables, hashtag pages, mention linking	hashtags	Parser + hashtag page	D3	Parser unit tests (case, punctuation, duplicates, unknown user)	M4 lead; M5 linkified text + page; M2 hooks into post creation
9	**Trending.** Repeatable job every ~5 min, last 24 h, cache in Redis	trending, jobs	`/trending/hashtags`; sidebar widget	D8	Window test (23 h vs 25 h), job rerun, cache TTL	M4 lead; M5 widget; M3 verifies job scheduling in deployment
10	**Private accounts.** `is_private`, follow requests, approve/reject, central `canView`	privacy, follows	Request flow; private profile UI	D4	Non-follower 403 on every read path; request lifecycle	M1 lead; M5 UI; M2 feed privacy checks; M4 search/hashtag privacy
11	**Notifications.** Create for like/reply/mention/follow, grouped read API, live badge	notifications	Notification service + page	D7, D8, D10	Grouping ("Rahul and 5 others"), no self-notifications, unlike removes row	M3 lead; M5 UI; M4 mention events; M1 follow events
12	**AI moderation.** `moderate-post` job, provider interface, flag + queue, moderator endpoints/page	moderation	Queue API and page; mock + real provider	D3, D6 (jobs)	Mock-LLM tests: flagged, safe, timeout, retries, manual review	M4 lead; M5 moderator page; M1 role guard; M2 hides flagged posts from feed
13	**Search, block and mute.** User/post search, block/mute APIs, enforcement everywhere	search, blocks	Search page; block/mute actions	D10	Blocked users absent from search/feed/interactions; mute hides notifications	M4 search; M1 block/mute API; M2 feed filters; M3 notification filters; M5 UI

Day	Objective and tasks	Modules	Deliverables	Depends on	Testing	Team
14	**Reliability.** Posting rate limit, test gaps, load test, README, bug fixing	all	Rate limiter, coverage report, README, seed script	all	429 tests, full E2E flow, k6 report, security checklist	M2 rate limiter; M3 CI hardening; all write tests; M5 E2E; M1 README
15	**Completion.** Deploy, smoke test, demo rehearsal, viva practice	all	Live URL, demo script, final docs	D14	Production smoke test, full demo run-through	M3 deploy; everyone demos their module; M1 coordinates

10. Beginner-Friendly MCA Viva Preparation

10.1 Core Explanations

Project in simple English

"Our project is a mini Twitter. People can create accounts, write short posts with pictures, follow others, and see a feed of their friends' posts. They can like, repost, reply, search, and get notifications. We made the feed fast using Redis, and live updates using Socket.IO."

Problem statement

"When millions of people read feeds, calculating the feed every time from the database is slow. We prepare the feed in advance so that opening the app is fast. We also needed safety features like private accounts, blocking, and abuse detection."

Why each technology

Technology	Simple reason
TypeScript	JavaScript with types, so fewer bugs
React	Builds the UI from reusable components
Express	Simple way to write the REST API
PostgreSQL	Safe, relational storage with unique constraints and transactions
Prisma	Writes DB queries in TypeScript and manages migrations
Redis	In-memory, so extremely fast; used for feeds, rate limits, queues
Docker	Everyone runs the same Postgres/Redis versions
Socket.IO	The server can push live updates to the browser
BullMQ	Runs slow tasks in the background with retries
LLM	Understands the meaning of text, so it detects abuse better than keyword lists
GitHub Actions	Automatically tests every pull request

Architecture

The browser talks to the API (REST) and to the socket server. The API reads and writes PostgreSQL. Redis holds feeds and queues. A worker process does background jobs such as fan-out, trending, and AI moderation.

Database

Users create posts. A reply is also a post that points to a parent. Follows, likes, and reposts are "join tables" with a composite primary key, so duplicates are impossible. Hashtags are stored once and linked to posts through `post_hashtags`.

Redis and fan-out on write

"Fan-out on write means work at posting time so reading is cheap. When Asha posts, we copy the post ID to each follower's feed list in Redis. When a follower opens the app, we just read their list. The trade-off is that posting by a user with very many followers costs more, so we do it in a background job."

Socket.IO

"Normal HTTP is request-reply. Socket.IO keeps a connection open so the server can send events. When someone likes a post, the server sends `post:counts` to everyone viewing it."

Cursor-based pagination

"Instead of 'page 2', we say 'give me 20 posts older than this post'. If new posts arrive while scrolling, page numbers would shift and show duplicates, but a cursor always continues from the same point."

Background jobs

"Jobs are tasks done outside the user's request, like sending the post to followers or running AI checks. They are stored in a Redis queue, and a worker processes them with retries if something fails. Trending runs as a repeating job every few minutes."

AI moderation

"After a post is saved, a job sends its text to an LLM and asks if it is abusive. If yes, the post is hidden and added to a queue for a human moderator. If the LLM is down, we retry and finally send it for manual review, so posting is never blocked."

GitHub workflow

"We never push to main. Each feature has a branch, we open a pull request, a teammate reviews it, automated tests run, and only then it is merged."

10.2 Thirty Likely Viva Questions

Q1. What is your project?

A micro-blogging social network with a fast Redis-powered home feed.

Q2. What is the main technical challenge?

Making the feed fast and keeping counts correct when many users act at once.

Q3. What is fan-out on write?

Copying a new post's ID into every follower's feed at posting time, so reading the feed is fast.

Q4. What is fan-out on read, and why not use it?

Building the feed by querying all followed users when it is opened. It is simpler but gets slow with many follows.

Q5. Why Redis for the feed?

It is in memory, so reads take milliseconds, and sorted sets keep posts ordered.

Q6. What do you store in the Redis feed?

Only post IDs, so deletes and count changes stay correct.

Q7. What if Redis crashes?

The feed falls back to PostgreSQL, a bit slower. Redis can be rebuilt from the database.

Q8. Why PostgreSQL and not MongoDB?

Our data is relational (users, follows, likes) and needs unique constraints and transactions.

Q9. What is a primary key vs a foreign key?

A primary key uniquely identifies a row. A foreign key points to a row in another table.

Q10. How do you ensure unique usernames?

Validation plus a unique index in the database, which handles two signups at the same instant.

Q11. How do you keep follower counts correct under concurrency?

Insert the follow row and increment the counter in one transaction, only if the insert really happened. A composite primary key blocks duplicates.

Q12. What is a transaction?

A group of database operations that all succeed or all fail.

Q13. What is idempotency?

Doing the same request twice has the same effect as doing it once. Liking twice does not add two likes.

Q14. What is cursor-based pagination?

"Give me items older than this one", which avoids duplicates and gaps.

Q15. Why not OFFSET pagination?

OFFSET is slower on large tables and shifts when new posts arrive.

Q16. How are replies stored?

As posts with `parent_id` (and `root_id` for the thread).

Q17. How do hashtags work?

We extract `#words` with a parser, lowercase them, store them once, and link them through `post_hashtags`.

Q18. How is trending computed?

A repeating job counts hashtag uses in the last 24 hours and caches the top list in Redis.

Q19. Why a background job for trending?

Counting is expensive. Doing it every few minutes keeps requests fast.

Q20. What is Socket.IO?

A library for real-time two-way communication between browser and server.

Q21. What are Socket.IO rooms?

Groups of connections. We use a room per post and per user to send targeted events.

Q22. What happens if the socket disconnects?

It auto-reconnects, and the client refetches current counts through REST.

Q23. How does a private account work?

Posts are visible only to approved followers. Others send a follow request that the owner approves.

Q24. Difference between block and mute?

Block removes interaction both ways. Mute only hides that user for me.

Q25. How are notifications grouped?

One row per event is stored. When reading, we group by type and post and show "X and N others".

Q26. How does AI moderation work, and what if it fails?

A job asks an LLM to classify the text. Flagged posts go to a queue. On failures we retry and then flag for manual review.

Q27. Why do you check posts in the background?

The LLM is slow and can fail, so we do not make users wait.

Q28. How do you prevent spam?

Rate limiting posts per user using Redis counters, returning HTTP 429.

Q29. How do you secure passwords and sessions?

Salted hashing (argon2/bcrypt), short-lived JWTs, httpOnly refresh cookies.

Q30. How do you prevent XSS and SQL injection?

React escapes output, we never insert raw HTML, and Prisma uses parameterized queries with Zod validation.

Be ready for follow-ups such as "What is Docker?", "What is CI?", "Why TypeScript?". The answers in 10.1 cover them.

10.3 Individual Contribution Template

*My name is **[name]** and I worked on **[module]**.*

What I built: *[2–3 features, e.g., follow API, counters, private accounts].*

How it works: *[step by step, e.g., request → validation → DB transaction → response].*

Technologies I used and why: *[e.g., Prisma for queries, Redis for …].*

A problem I faced and how I solved it: *[e.g., duplicate follows under concurrency → composite primary key plus transactional counters].*

How I tested it: *[unit / API / concurrency tests].*

My GitHub contribution: *[folders, PRs, number of commits].*

How my module connects to others: *[e.g., my follow events create notifications for M3's service].*

Fill-in hints for each member: use the "Viva topics" row in Section 3.

Assumptions

- Web app only; English UI; email + password login.
- Text limit 280 characters; up to 4 images; fan-out cap of 800 IDs.
- Team of 5 with roles as in Section 3.
- Free-tier hosting is acceptable.

Decisions the Team Must Approve Before Development

#	Decision	Recommendation
1	Frontend framework	React + Vite + TanStack Query (vs Next.js)
2	Backend framework	Express + Zod (vs Fastify/NestJS)
3	Database and ORM	PostgreSQL + Prisma
4	Redis feed structure	Sorted set per user, capped at 800 (vs plain List)
5	Auth approach	JWT access + httpOnly refresh cookie, argon2 hashing; login by email or username
6	Image storage	Cloudinary or R2 (local disk in dev)
7	LLM provider and model for moderation	Choose one with a free/cheap tier; adapter interface; define abuse categories and flag threshold
8	Moderation timing	Publish-then-check (hide if flagged) vs hold-until-checked
9	Do reposts appear in followers' feeds?	MVP: show on the reposter's profile and count only; feed entries as a later upgrade
10	Private → public switch	Auto-approve pending requests
11	Post limits	280 chars, 4 images, 5 MB each; rate limit 10 posts/min
12	Notification grouping	Store per event, group on read
13	Block behaviour	Remove follows both ways; hide each other everywhere
14	User "Report" button	Treat as Optional P2
15	Deployment hosts	Vercel + Render/Railway + Neon + Upstash
16	Team role assignment	Confirm M1–M5 names and swap if needed
17	Git rules	Branch protection, 1 reviewer, squash merge, Conventional Commits
18	Moderator account creation	Seed script sets `role = moderator` (no admin UI)